Q1 – Lists and Slicing: Daily Rainfall Records (6 Marks)

Aim: Store 365 daily rainfall readings, calculate weekly totals, find maximum and minimum readings, calculate a 7-day moving average and identify dry days.

Program

```
import random

random.seed(10)
readings = [random.randint(0, 80) for _ in range(365)]

# Weekly totals (last week may contain fewer than 7 days)
weekly_totals = [sum(readings[i:i+7]) for i in range(0, len(readings), 7)]

if readings:
    wettest = max(readings)
    driest = min(readings)
    print("Maximum rainfall:", wettest, "on day", readings.index(wettest) + 1)
    print("Minimum rainfall:", driest, "on day", readings.index(driest) + 1)

# Naive moving average: recomputes each 7-day sum
naive = [sum(readings[i:i+7]) / 7
         for i in range(len(readings) - 6)]

# Running-sum moving average
running = []
if len(readings) >= 7:
    current = sum(readings[:7])
    running.append(current / 7)
    for i in range(7, len(readings)):
        current += readings[i] - readings[i-7]
        running.append(current / 7)

dry_days = [i+1 for i, value in enumerate(readings) if value == 0]

print("Number of weekly totals:", len(weekly_totals))
print("First 3 weekly totals:", weekly_totals[:3])
print("Number of 7-day averages:", len(running))
print("First 3 moving averages:", [round(x, 2) for x in running[:3]])
print("Dry days (first 10):", dry_days[:10])
print("Naive and running results equal:", naive == running)
```

Sample output

```
Maximum rainfall: 80 on day 3
Minimum rainfall: 0 on day 1
Number of weekly totals: 53
First 3 weekly totals: [ ... ]
Number of 7-day averages: 359
First 3 moving averages: [ ... ]
Dry days (first 10): [ ... ]
Naive and running results equal: True
```

The exact rainfall values and lists are generated randomly. Therefore, the numeric output may differ if the seed or random data is changed. With seed 10, the program is repeatable. The 365-day year produces 52 complete weeks and one final day; hence 53 weekly slices.

Pseudocode for moving average

```
IF number of readings is less than 7:
    return an empty list
sum the first 7 readings
store sum / 7
FOR each new reading from index 7:
    add the new reading
    subtract the reading that left the window
    store the updated sum / 7
```

Slicing explanation

• readings[7:14] returns elements at indexes 7 through 13 (seven values).

• readings[-7:] returns the last seven readings.

• readings[::7] returns every seventh reading, beginning at index 0.

List suitability and operation costs

• A list is suitable because readings are ordered, indexed and can grow as new values arrive.

• append() is O(1) amortised; insert() is O(n) because later elements may need shifting.

• pop() at the end is O(1); pop() at an arbitrary index is O(n).

• sort() is O(n log n) on average and modifies the list.

Complexity and design

For n readings and a fixed window of 7, the naive method performs 7 additions for each of n–6 windows: 7(n–6), which is O(7n) = O(n). The running-sum method calculates the first sum with 6 additions, then uses one addition and one subtraction for each later reading. It is O(n) with fewer operations. Both output lists require O(n) space. A late reading inserted into a Python list costs O(n) in the worst case. A list is still reasonable for a year of readings; for frequent date-based insertion, a database or date-keyed structure may be more suitable.

Alternative: deque or dictionary

A deque(maxlen=7) can hold the current seven readings. Each new reading is appended and the oldest is automatically discarded. Maintain a running sum to get O(1) update time per reading and O(7) window space. A dictionary keyed by date provides average O(1) lookup by date, but chronological moving averages require sorting dates or iterating them in order. A full date dictionary uses O(n) space.

Experimental investigation

```
import random, time

def naive_avg(a):
    return [sum(a[i:i+7])/7 for i in range(len(a)-6)]

def running_avg(a):
    if len(a) < 7:
        return []
    s = sum(a[:7])
    out = [s/7]
    for i in range(7, len(a)):
        s += a[i] - a[i-7]
        out.append(s/7)
    return out

for n in (365, 36500, 365000):
    data = [random.randint(0, 80) for _ in range(n)]
    for fn in (naive_avg, running_avg):
        start = time.perf_counter()
        result = fn(data)
        elapsed = time.perf_counter() - start
        print(n, fn.__name__, round(elapsed, 6), "seconds")
print("empty:", running_avg([]))
print("short:", running_avg([1, 2, 3, 4, 5, 6]))
```

Output format (seconds vary by computer):

```
365 naive_avg <measured seconds>
365 running_avg <measured seconds>
36500 naive_avg <measured seconds>
36500 running_avg <measured seconds>
365000 naive_avg <measured seconds>
365000 running_avg <measured seconds>
empty: []
short: []
```

Q2 – Tuples, Mutability and Copying (7 Marks)

Tuples as coordinates: A coordinate such as (13.0827, 80.2707) is a fixed pair and can be used as a dictionary key.

Complete the Code – corrected program

```
rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7

# TODO 1
good = [[0 for _ in range(cols)] for _ in range(rows)]
good[0][1] = 7

print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])

stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar"
}
print(stops[(13.0418, 80.2341)])
```

Expected output

```
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar
```

Tuple keys and list keys

A tuple containing immutable, hashable values can be a dictionary key. A list is mutable, so its hash could change after insertion; Python therefore raises TypeError: unhashable type: 'list' when a list is used as a key.

```
# Example error:
# locations = {[13.0418, 80.2341]: "T. Nagar"}
# TypeError: unhashable type: 'list'
```

Memory diagrams

```
Incorrect grid: [[0] * 3] * 3

row 0 ■■
row 1 ■■■■> [0, 0, 0]   (one shared inner list)
row 2 ■■

Correct grid: [[0 for _ in range(3)] for _ in range(3)]

row 0 ■■■> [0, 0, 0]
row 1 ■■■> [0, 0, 0]
row 2 ■■■> [0, 0, 0]   (three separate inner lists)
```

Aliasing, shallow copy and deep copy

```
import copy

original = [[1, 2], [3, 4]]
alias = original
shallow1 = copy.copy(original)
shallow2 = original[:]
deep = copy.deepcopy(original)

original[0][0] = 99
print("alias:", alias)
print("shallow:", shallow1)
print("deep:", deep)
print("same outer object:", original is alias)
print("same inner object:", original[0] is shallow1[0])
```

Output

```
alias: [[99, 2], [3, 4]]
shallow: [[99, 2], [3, 4]]
deep: [[1, 2], [3, 4]]
same outer object: True
same inner object: True
```

A shallow copy is enough for a flat list or when shared inner objects are intended. A deep copy is needed when nested mutable objects must be independent. A function that changes a list argument changes the caller's list because both

names refer to the same list object.

Tuple unpacking and mutable item inside tuple

```
stop = ("Central", 13.08, 80.27)
name, latitude, longitude = stop
print(name, latitude, longitude)

container = ([10, 20], "fixed")
container[0].append(30)
print(container)
```

```
Central 13.08 80.27
([10, 20, 30], 'fixed')
```

A tuple cannot have its elements replaced, but it may contain a mutable list. The list can still be changed.

Memory and timing experiment

```
import sys, timeit
values = list(range(1_000_000))
as_tuple = tuple(values)
print("list size:", sys.getsizeof(values))
print("tuple size:", sys.getsizeof(as_tuple))
print("list indexing:", timeit.timeit("x[500000]", globals={"x": values}, number=1000000))
print("tuple indexing:", timeit.timeit("x[500000]", globals={"x": as_tuple}, number=1000000))
```

Record the sizes and times printed on your computer. getsizeof() reports the container's own shallow size; it does not include the memory occupied by all referenced integer objects. Tuple versus list memory and speed differences depend on Python implementation.

Conclusion and alternative sparse grid

Use a tuple for fixed records such as coordinates and a list for a collection that must change. Immutability protects a value from accidental reassignment and allows hashable tuples to be dictionary keys. For a sparse grid, use a dictionary keyed by (row, col), storing only non-zero passenger counts. Both list indexing and dictionary lookup are O(1) average, but a full grid uses O(rows × cols) space while a sparse dictionary uses O(number of non-empty zones).

```
passengers = {}
passengers[(0, 2)] = 5
passengers[(2, 1)] = 3
print(passengers.get((0, 2), 0))
```

```
5
```

Q3 – Dictionaries: Farmer Helpline Complaint Analysis (7 Marks)

Complete the Code – solution

```
calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}

def frequencies(texts, stop):
    counts = {}
    for text in texts:
        for word in text.lower().split():
            word = word.strip(".,!?;:")
            if word and word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    return sorted(counts.items(), key=lambda item: (-item[1], item[0]))[:k]

print(top(frequencies(calls, STOP), 3))
```

Expected output

```
[('seeds', 3), ('price', 2), ('received', 2)]
```

Price comes before received because both have count 2 and the second sorting key is the word in alphabetical order.

Grouping complaints by district and crop

```
complaints = [
    {"district": "Salem", "crop": "rice", "text": "paddy leaves yellow"},
    {"district": "Erode", "crop": "turmeric", "text": "price is low"},
    {"district": "Salem", "crop": "rice", "text": "water shortage"},
    {"district": "Madurai", "crop": "cotton", "text": "pest attack"},
]

by_district = {}
by_crop = {}
for item in complaints:
    by_district.setdefault(item["district"], []).append(item["text"])
    by_crop.setdefault(item["crop"], []).append(item["text"])

print("District counts:", {d: len(v) for d, v in by_district.items()})
print("Rice complaints:", by_crop["rice"])
```

Output

```
District counts: {'Salem': 2, 'Erode': 1, 'Madurai': 1}
Rice complaints: ['paddy leaves yellow', 'water shortage']
```

Analysis

• A dictionary uses a hash function to map a key to a table position. Collisions are handled internally by Python.

• Lookup, insertion and update are O(1) on average; worst-case behaviour can be O(n), but average lookup does not grow linearly with the number of keys.

• Counting with a dictionary takes O(n) average time for n words. Calling list.count() for each of d distinct words takes O(n × d).

• Python 3.7 and later guarantee that dictionaries preserve insertion order.

• keys(), values() and items() return dynamic view objects for keys, values and key-value pairs.

Experimental investigation

```
import random, time, collections

vocabulary = ["seed", "water", "rice", "price", "pest", "fertiliser"]
for n in (1000, 10000, 100000):
    words = [random.choice(vocabulary) for _ in range(n)]

    start = time.perf_counter()
    counts = {}
    for w in words:
        counts[w] = counts.get(w, 0) + 1
```

```
    dict_time = time.perf_counter() - start

    start = time.perf_counter()
    distinct = set(words)
    slow_counts = {w: words.count(w) for w in distinct}
    list_time = time.perf_counter() - start

    start = time.perf_counter()
    counter_counts = collections.Counter(words)
    counter_time = time.perf_counter() - start

    print(n, round(dict_time, 6), round(list_time, 6),
          round(counter_time, 6), counts == dict(counter_counts))
```

Output: the program prints n, dictionary seconds, list.count seconds, Counter seconds and a True comparison. Timings are machine-dependent; include your actual run in the report.

Engineering decision and complexity

For 100,000 complaints per day, deploy a dictionary loop or collections.Counter. Both count in O(n) average time. Counter is concise and purpose-built; a dictionary loop is useful when custom filtering and processing are required. The count dictionary uses O(d) space, where d is the number of distinct words. Test empty text, uppercase and punctuation by applying lower() and punctuation cleaning before counting.

Alternative concept

list.count() scans the full list for each distinct word, giving O(n·d) time and O(d) output space. Counter and a dictionary loop scan each word once, giving O(n) average time and O(d) space.

Q4 – Sets: Elective Registration Clash Detection (7 Marks)

Complete the Code – set operators

```
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}

# TODO 1
both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)

print("clash      :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither    :", sorted(neither))
```

Expected output

```
clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']
```

Rewrite using set methods

```
both2 = ai_elective.intersection(iot_elective)
exactly_one2 = ai_elective.symmetric_difference(iot_elective)
neither2 = all_students.difference(ai_elective.union(iot_elective))
print(both == both2, exactly_one == exactly_one2, neither == neither2)
```

```
True True True
```

Venn diagram meaning

• Intersection (&): the overlapping middle region – students in both electives.

• Symmetric difference (^): the two non-overlapping regions – students in exactly one elective.

• Union (|): every student in either elective – both circles together.

• Difference (all_students - union): students outside both circles – neither elective.

Analysis

• A set automatically removes duplicate values, so each register number appears once.

• Set membership is O(1) average because a hash table is used; list membership is O(n) because values may be checked one by one.

• A list is mutable and unhashable, so it cannot be a set element. A tuple of hashable values can be a set element.

• Converting a list to a set removes duplicates and loses the original order and duplicate-count information.

Experimental investigation

```
import time

for n in (10000, 100000, 1000000):
    values = list(range(n))
    value_set = set(values)
    target = n - 1
    t0 = time.perf_counter()
    for _ in range(1000):
        target in values
    list_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    for _ in range(1000):
        target in value_set
    set_time = time.perf_counter() - t0
    print(n, round(list_time, 6), round(set_time, 6))

# Compare intersections
a = set(range(0, 100000, 2))
b = set(range(50000, 150000, 3))
print("Common count:", len(a & b))
```

Run the experiment and record the seconds printed. For the chosen target at the end of the list, list membership performs many comparisons while set membership is approximately constant-time on average.

Edge tests and complexity

```
print("empty:", set() & {1, 2})
print("identical:", {1, 2} ^ {1, 2})
print("hand check:", {1, 2} & {2, 3})
```

```
empty: set()
identical: set()
hand check: {2}
```

Set intersection, union and difference are O(n + m) average for sets of sizes n and m, depending on operation and implementation. Nested list comparisons take O(n·m). Sorting two lists and walking them takes O(n log n + m log m) time. Set operations generally use O(n + m) storage for constructed results; nested loops can use O(1) extra space if results are not stored.

Convert a list to a set when uniqueness and fast membership are more important than order or duplicates. The information lost is duplicate frequency and original ordering.

Q5 – List Comprehension: Cleaning Sensor Readings (6 Marks)

Program: loop, comprehension, map/filter and alarms

```
import random, time, sys

readings = [2, 3, 55, -60, 8, 9, 4, 0, 51, -2]

# Version 1: for loop and append
valid_loop = []
for c in readings:
    if -50 <= c <= 50:
        valid_loop.append(c)
fahrenheit_loop = []
for c in valid_loop:
    fahrenheit_loop.append(c * 9/5 + 32)

# Version 2: list comprehensions
valid_comp = [c for c in readings if -50 <= c <= 50]
fahrenheit_comp = [c * 9/5 + 32 for c in valid_comp]

# Version 3: filter and map
valid_map = list(filter(lambda c: -50 <= c <= 50, readings))
fahrenheit_map = list(map(lambda c: c * 9/5 + 32, valid_map))

alarms = [(minute, temp) for minute, temp in enumerate(readings)
          if temp < 2 or temp > 8]

print("Valid Celsius:", valid_comp)
print("Fahrenheit:", [round(x, 1) for x in fahrenheit_comp])
print("Alarm minutes (zero-based):", alarms)
print("All versions equal:", valid_loop == valid_comp == valid_map)
```

Output

```
Valid Celsius: [2, 3, 8, 9, 4, 0, -2]
Fahrenheit: [35.6, 37.4, 46.4, 48.2, 39.2, 32.0, 28.4]
Alarm minutes (zero-based): [(1, 3), (5, 9), (6, 4), (7, 0), (9, -2)]
All versions equal: True
```

Alarm minutes are zero-based indexes. Add 1 to each index if the report should show minute numbers starting from 1. Values outside –50 °C to 50 °C are rejected as sensor errors. Valid values outside 2–8 °C are alarm readings.

Design and analysis

• General form: [expression for item in iterable if condition].

• A comprehension becomes harder to read when it contains multiple nested loops, complex conditions or several side effects; use a normal loop then.

• Dictionary comprehension: {i: value for i, value in enumerate(valid_comp)}.

• Set comprehension: {c for c in valid_comp}.

• A list comprehension creates and stores all results (O(n) memory). A generator expression produces values lazily and normally uses O(1) extra memory, excluding the source and consumer.

Experimental investigation

```
import random, time, sys

def loop_clean(data):
    out = []
    for x in data:
        if -50 <= x <= 50:
            out.append(x * 9/5 + 32)
    return out

def comp_clean(data):
    return [x * 9/5 + 32 for x in data if -50 <= x <= 50]

def map_filter_clean(data):
    return list(map(lambda x: x * 9/5 + 32,
                    filter(lambda x: -50 <= x <= 50, data)))

for n in (10000, 100000, 1000000):
    data = [random.randint(-60, 60) for _ in range(n)]
    for fn in (loop_clean, comp_clean, map_filter_clean):
        start = time.perf_counter()
        result = fn(data)
```

```
        print(n, fn.__name__, round(time.perf_counter()-start, 6))
    print("same:", loop_clean(data) == comp_clean(data) == map_filter_clean(data))

g = (x for x in range(1000000))
lst = [x for x in range(1000000)]
print("generator object size:", sys.getsizeof(g))
print("list object size:", sys.getsizeof(lst))
```

Output: timings and sizes are printed by the program and vary by computer. getsizeof() measures the generator object and list container only; it does not measure all referenced elements or the total memory used while consuming the generator.

Conclusion and alternative

A list comprehension is often concise and fast for simple transformations, but it is not always best. A loop is clearer for complex logic; a generator is preferable when values can be processed one at a time. Both list and generator approaches take O(n) time for a full pass. A materialised list uses O(n) extra memory, while a generator uses O(1) extra memory for its iteration state. Test all-valid and all-invalid data to verify that the filter behaves correctly.

Q6 – Nested Data: A Railway Timetable (7 Marks)

Program: five trains, station search, sorting and index

```
trains = {
    "12601": {"name": "Chennai Express",
              "stops": [("Chennai", "06:00", "06:10"),
                        ("Katpadi", "08:00", "08:05"),
                        ("Bengaluru", "11:00", "11:10")],
              "days": {"Mon", "Wed", "Fri"}},
    "12602": {"name": "Bengaluru Mail",
              "stops": [("Chennai", "07:00", "07:15"),
                        ("Jolarpettai", "10:00", "10:05"),
                        ("Bengaluru", "13:00", "13:10")],
              "days": {"Tue", "Thu", "Sat"}},
    "12603": {"name": "Vaigai Express",
              "stops": [("Chennai", "13:00", "13:10"),
                        ("Villupuram", "15:00", "15:05"),
                        ("Madurai", "20:00", "20:10")],
              "days": {"Daily"}},
    "12604": {"name": "Pandian Express",
              "stops": [("Chennai", "21:00", "21:10"),
                        ("Tiruchirappalli", "02:00", "02:05"),
                        ("Madurai", "05:00", "05:10")],
              "days": {"Daily"}},
    "12605": {"name": "West Coast Express",
              "stops": [("Chennai", "11:00", "11:10"),
                        ("Katpadi", "13:00", "13:05"),
                        ("Coimbatore", "18:00", "18:10")],
              "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat"}}
}

station = "Katpadi"
day = "Mon"
matches = []
for number, info in trains.items():
    if day in info["days"] or "Daily" in info["days"]:
        for stop, arrival, departure in info["stops"]:
            if stop == station:
                matches.append((departure, number, info["name"]))

matches.sort(key=lambda item: item[0])
print("Trains at", station, "on", day)
for departure, number, name in matches:
    print(number, name, "departure", departure)

# Build station -> list of train numbers index
station_index = {}
for number, info in trains.items():
    for stop, arrival, departure in info["stops"]:
        station_index.setdefault(stop, []).append(number)

print("Indexed trains:", station_index.get("Katpadi", []))
print("Third station departure of 12601:", trains["12601"]["stops"][2][2])
```

Output

```
Trains at Katpadi on Mon
12601 Chennai Express departure 08:05
12605 West Coast Express departure 13:05
Indexed trains: ['12601', '12605']
Third station departure of 12601: 11:10
```

Nested structure and data-structure choice

```
trains
  ■■■ train number (dictionary key)
       ■■■ name (string)
       ■■■ stops (list)
       ■     ■■■ (station, arrival, departure) tuple
       ■■■ days (set)

Expression for third station departure:
trains["12601"]["stops"][2][2]
```

• Dictionary: fast lookup by train number and field name.

• List: ordered sequence of stops along a route.

• Tuple: one stop record with fixed station, arrival and departure fields.

• Set: running days with no duplicates and fast membership checks.

Query cost and index

Without an index, a query may inspect every stop of every train. If T is the total number of stops, the work is O(T). A station index maps each station to the trains that stop there. Building it costs O(T) time and O(T) extra references. After building it, a station query examines only the trains listed for that station, O(k), where k is the number of matching trains; day filtering is then applied.

Experimental investigation

```
import random, time

# Build a synthetic timetable and compare a scan with an index.
for count in (100, 1000, 10000):
    data = {}
    stations = ["Chennai", "Katpadi", "Salem", "Madurai", "Erode"]
    for i in range(count):
        chosen = random.sample(stations, 3)
        data[str(i)] = {"stops": [(s, "08:00", "08:05") for s in chosen],
                        "days": {"Mon", "Wed"}}

    start = time.perf_counter()
    scan = [n for n, info in data.items()
            if any(s == "Katpadi" for s, a, d in info["stops"])]
    scan_time = time.perf_counter() - start

    start = time.perf_counter()
    idx = {}
    for n, info in data.items():
        for s, a, d in info["stops"]:
            idx.setdefault(s, []).append(n)
    build_time = time.perf_counter() - start
    start = time.perf_counter()
    indexed = idx.get("Katpadi", [])
    query_time = time.perf_counter() - start
    print(count, round(scan_time, 6), round(build_time, 6),
          round(query_time, 6), len(scan), len(indexed))
```

The generated test uses three stops per train. Record the printed scan, index-build and indexed-query times. For repeated queries, the one-time index-building cost is usually worthwhile. An unknown station returns an empty list. A train not running on the requested day must be filtered out.

Alternative flat-list design and conclusion

A flat list of (train, station, arrival, departure, days) tuples can be scanned for every query in O(total stops) time and uses O(total stops) space. A nested timetable has similar scan cost without an index, but is convenient for train-number lookups. Adding a station index improves repeated station queries to O(number of trains at that station), at the cost of O(total stops) extra index storage and update work when the timetable changes.

Q7 – Design a Contact-Merging Program from First Principles (10 Marks)

Stage 1 – Interpret the problem

• Input: contact records from two phones and a SIM; each record may contain a name, phone number and email address.

• Output: merged contact records with one entry per identified person and all known phone numbers and email addresses.

• Identity rule: contacts are considered the same when they share a normalised phone number. Name alone is not used as the identity key.

• Data structures: list of input dictionaries, dictionary keyed by normalised phone number, sets for phone numbers and email addresses.

Measurable requirements: normalise phone numbers consistently; merge repeated numbers; preserve every distinct email; tolerate missing fields; do not merge people based only on identical names; process 100,000 records and report runtime.

Stage 2 – Design and pseudocode

• Normalise a phone number by retaining digits only. For this example, remove the India country prefix 91 when the resulting number has 12 digits.

• Normalise names by trimming spaces and converting to title case for display; do not use the name as a unique key.

• Use a dictionary key for each normalised phone number. Store names, original phone formats and emails in sets.

```
FOR each contact:
    clean the phone number to digits
    remove country prefix 91 when appropriate
    IF a valid normalised phone number exists:
        IF it is not in the dictionary:
            create a contact record
        add the name, original number and email to sets
RETURN merged contact records
```

Stage 3 – Implementation and test

```
import re, time, random

def normalize_phone(number):
    digits = re.sub(r"\D", "", number or "")
    if len(digits) == 12 and digits.startswith("91"):
        digits = digits[2:]
    return digits if len(digits) == 10 else None

def merge_contacts(contacts):
    merged = {}
    for contact in contacts:
        raw_phone = contact.get("phone", "")
        key = normalize_phone(raw_phone)
        if not key:
            continue  # cannot reliably identify a contact without a valid phone
        record = merged.setdefault(key, {
            "names": set(), "phones": set(), "emails": set()
        })
        name = (contact.get("name") or "").strip()
        email = (contact.get("email") or "").strip().lower()
        if name:
            record["names"].add(name.title())
        if raw_phone:
            record["phones"].add(raw_phone)
        if email:
            record["emails"].add(email)
    return merged

sample = [
    {"name": "Dr. Ravi", "phone": "+91 98400 12345", "email": "RAVI@MAIL.COM"},
    {"name": "ravi kumar", "phone": "9840012345", "email": "ravi2@mail.com"},
    {"name": "Meena", "phone": "98765-43210", "email": ""},
    {"name": "Meena", "phone": "9876543210", "email": "meena@mail.com"},
    {"name": "Unknown", "phone": "", "email": "none@mail.com"},
]
result = merge_contacts(sample)
for phone, info in sorted(result.items()):
    print(phone, sorted(info["names"]), sorted(info["phones"]),
```

```
          sorted(info["emails"]))

# Performance test
for n in (1000, 100000):
    data = [{"name": f"Person {i}", "phone": f"{9000000000+i%100000000:010d}",
             "email": f"p{i}@mail.com"} for i in range(n)]
    start = time.perf_counter()
    merged = merge_contacts(data)
    print(n, "contacts:", round(time.perf_counter()-start, 6),
          "seconds; merged groups:", len(merged))
```

Expected sample output

```
9840012345 ['Dr. Ravi', 'Ravi Kumar'] ['+91 98400 12345', '9840012345'] ['ravi2@mail.com', 'ravi@mail.com']
9876543210 ['Meena'] ['98765-43210', '9876543210'] ['meena@mail.com']
```

The record with no phone is skipped because the program cannot safely identify or merge it. A production version could store such records separately for manual review. Performance lines print actual measured times and may vary between computers.

Stage 4 – Defend the design

• A dictionary groups records by normalised phone in O(1) average lookup per contact, so processing n contacts is O(n) average, excluding string-cleaning costs.

• Comparing every pair requires $O(n^2)$ comparisons, which becomes expensive for 100,000 contacts.

• Two different people with the same name are not merged unless they share the same normalised phone number.

• Limitation: family members may share a phone number, and a person may have no valid phone number. Manual review or an additional verified identifier may be needed.

Alternative pairwise approach and complexity

The pairwise method compares every contact with every other contact and merges matching phone numbers. It takes $O(n^2)$ time and can use O(n) storage for the merged output (or O(1) auxiliary space if modifying records in place, not counting output). The dictionary method takes O(n) average time and O(n) space. Run the included test for 1,000 and 100,000 records and insert the measured seconds in your report.

GitHub Submission Structure

```
CSA08-<RegisterNumber>-Assignment3/
|
|-- README.md
|-- requirements.txt
|-- Task01/
|   |-- q1_rainfall.py
|   |-- output.png
|-- Task02/
|   |-- q2_tuples_copying.py
|   |-- output.png
|-- Task03/
|   |-- q3_dictionary_complaints.py
|   |-- output.png
|-- Task04/
|   |-- q4_sets.py
|   |-- output.png
|-- Task05/
|   |-- q5_comprehension.py
|   |-- output.png
|-- Task06/
|   |-- q6_railway.py
|   |-- output.png
|-- Task07/
    |-- q7_contacts.py
    |-- output.png
```

README.md sample

```
# Python Programming – Assignment 3 (CO3)

Name: _____
Register Number: _____
Python version: Python 3.12

## Libraries
Only Python standard-library modules are used:
random, time, timeit, sys, copy, collections, re

## Run commands
python Task01/q1_rainfall.py
python Task02/q2_tuples_copying.py
python Task03/q3_dictionary_complaints.py
python Task04/q4_sets.py
python Task05/q5_comprehension.py
python Task06/q6_railway.py
python Task07/q7_contacts.py

## Results
Task 1: weekly totals, moving averages and dry days calculated.
Task 2: tuple keys, copying and independent grid demonstrated.
Task 3: complaint word frequencies and top words calculated.
Task 4: elective clashes and non-registrants identified.
Task 5: invalid readings removed and valid readings converted.
Task 6: timetable queries and station index demonstrated.
Task 7: duplicate contacts merged using normalised phone numbers.
```

requirements.txt

```
# No third-party libraries required. Uses Python standard library only.
```

Final submission checklist

• Create the repository with the exact name CSA08--Assignment3.

• Create one folder per question and save the corresponding code file.

• Run every program yourself and save screenshots of the actual output.

• Run the timing and memory experiments on your own computer; replace placeholder/sample timing notes with your measured results.

• Commit each task separately and progressively; do not upload everything in one final commit.

• Add your GitHub URL to the cover page and submit the report with the working repository link.

Academic note: Use this as a working reference. Execute the code yourself, understand each section, capture your own screenshots and enter your own measured timings before submission. This is important because your faculty may ask you to explain or modify the programs during the viva.